

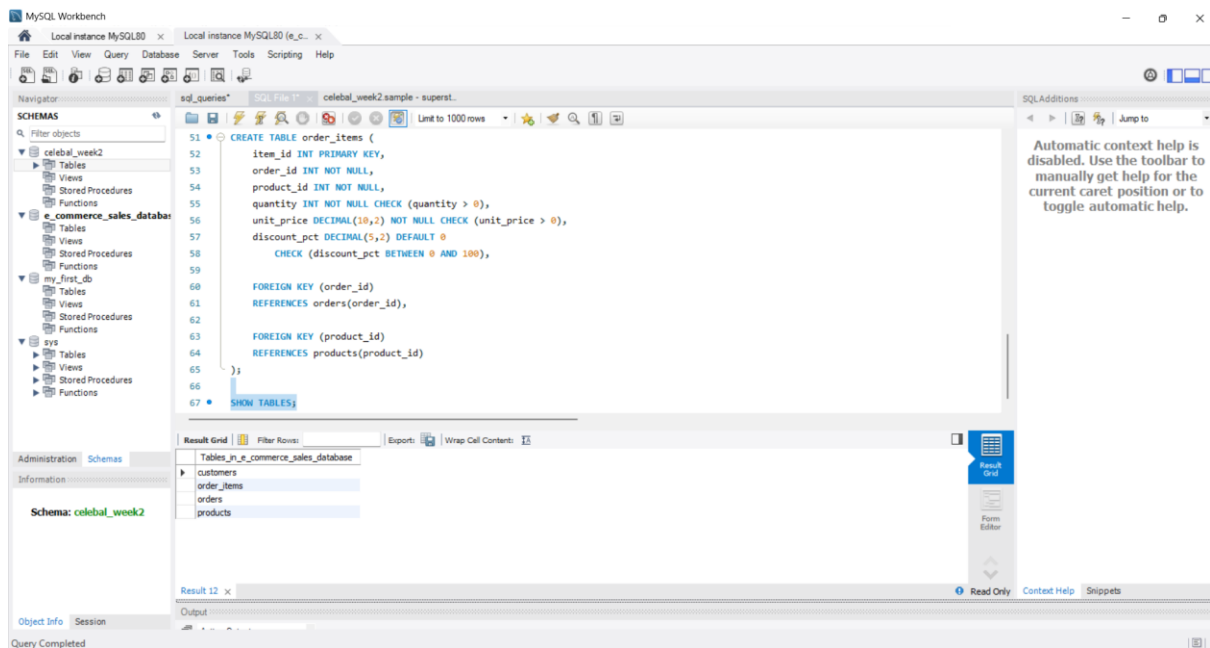
Week 2 Task: E-Commerce Sales Database

-Submitted By : Siga Nithin

Database and Table Creation :

A new database named **e_commerce_sales_database** was created in MySQL. Four tables — **customers**, **products**, **orders**, and **order_items** — were created based on the given schema. Primary Keys, Foreign Keys, and other constraints were added to maintain data integrity and relationships between tables.

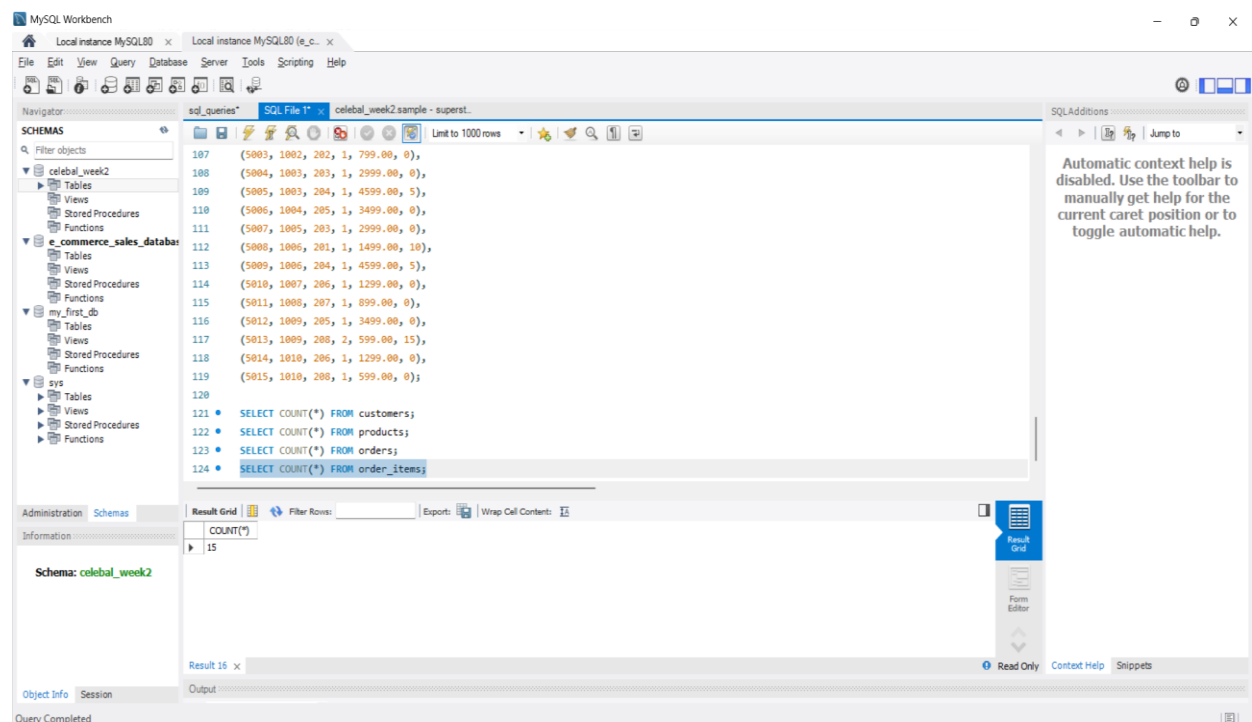
Output: Database and tables created successfully.



Data Insertion

After creating the tables, the sample data provided in the assignment was inserted into all four tables. The inserted data was verified using SQL queries to ensure that all records were loaded correctly and the database was ready for analysis.

Output: Sample data inserted and verified successfully.

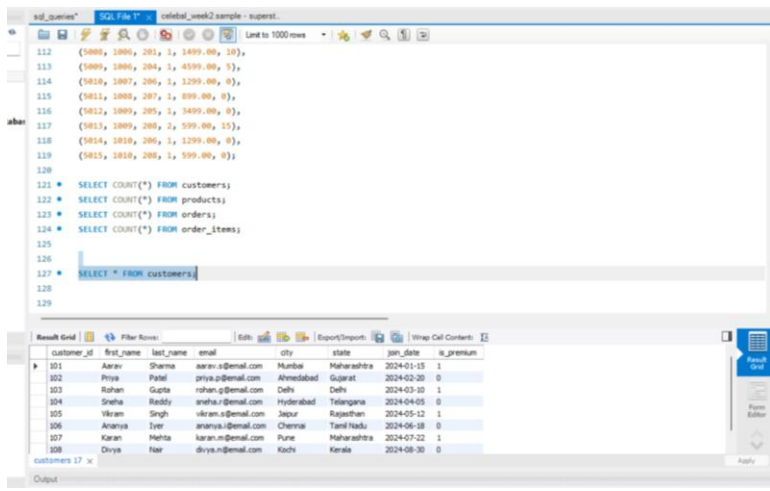


SQL Query Execution and Analysis :

Section A – SQL Basics

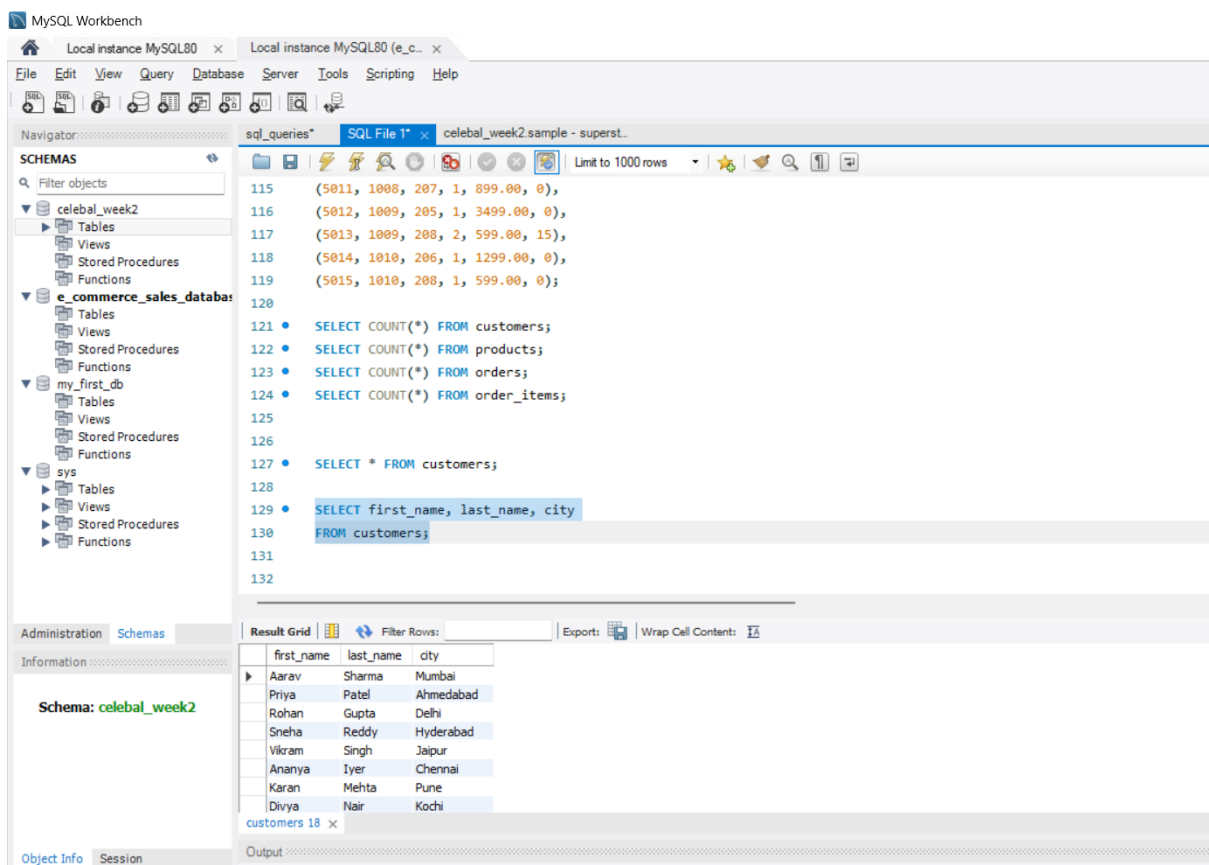
Q1. Display all columns and rows from the customers table.

Ans: SELECT * FROM customers;



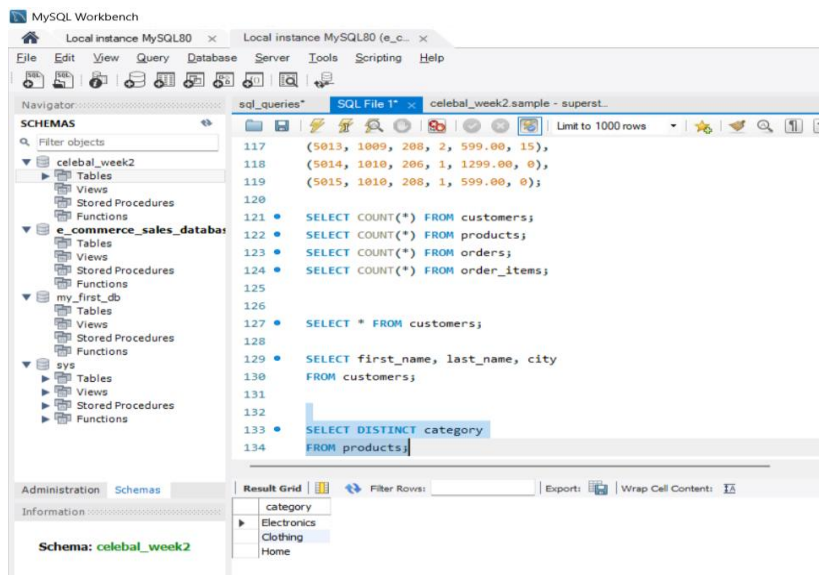
Q2. Retrieve only the first_name, last_name, and city of all customers.

Ans: **SELECT first_name, last_name, city**
FROM customers;



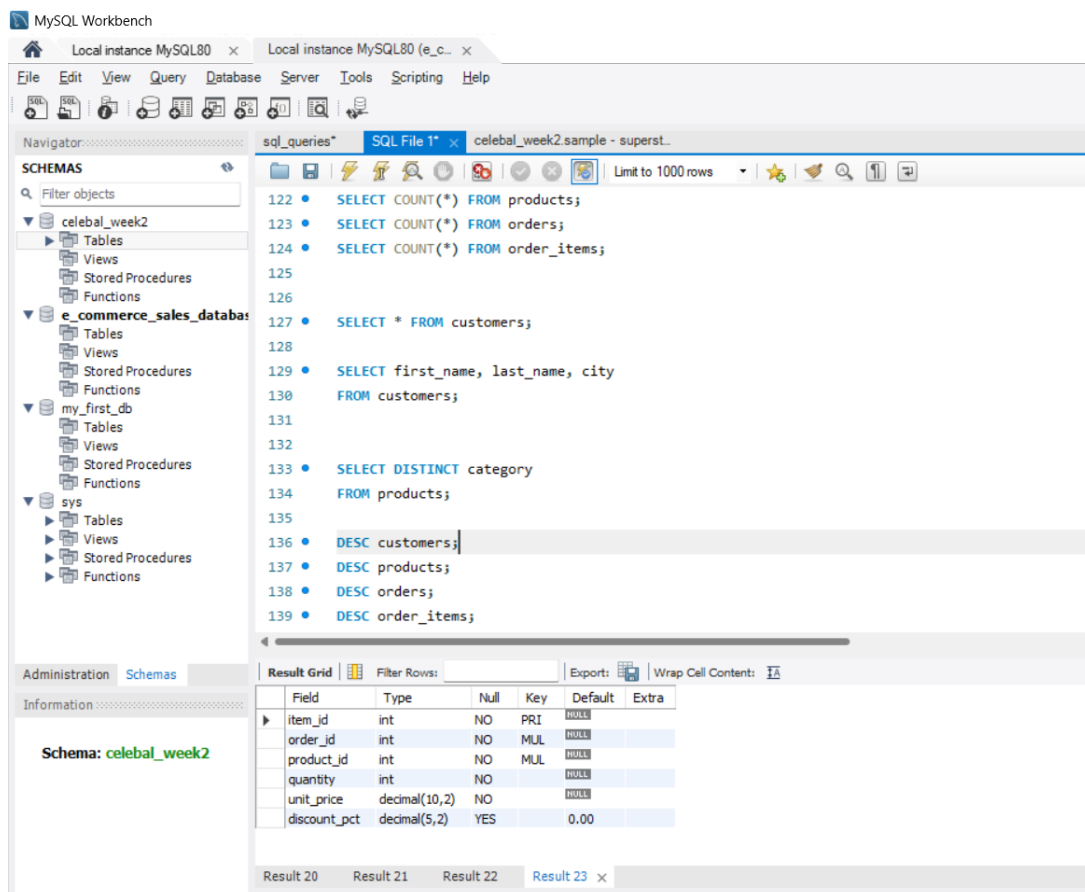
Q3. List all unique categories available in the products table.

Ans: **SELECT DISTINCT category**
FROM products;



Q4. Identify the Primary Key of each table. Explain why a Primary Key must be UNIQUE and NOT NULL.

Ans :



The Primary Key uniquely identifies each record in a table. It must be UNIQUE and NOT NULL to ensure data integrity and avoid duplicate records.

Q5. What constraints are applied to the email column in the customers table?

Ans : **DESC customers;**

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	
first_name	varchar(50)	NO		NULL	
last_name	varchar(50)	NO		NULL	
email	varchar(100)	NO	UNI	NULL	
city	varchar(50)	NO		NULL	
state	varchar(50)	NO	MUL	NULL	
join_date	date	NO		NULL	
is_premium	tinyint(1)	YES		0	

Q6. Try inserting a product with unit_price = -50.

The screenshot shows a SQL IDE interface. The top window displays a query with the following SQL code:

```
135
136 • DESC customers;
137 • DESC products;
138 • DESC orders;
139 • DESC order_items;
140
141
142 • DESC customers;
143
144
145 • INSERT INTO products
146   VALUES (
147     209,
148     'Test Product',
149     'Electronics',
150     'TestBrand',
151     -50,
152     100
153   );
```

The bottom window, titled 'Output', shows the execution results of the queries. It includes a table with columns: #, Time, Action, and Message.

#	Time	Action	Message
102	17:00:56	DESC products	6 row(s) returned
103	17:00:56	DESC orders	5 row(s) returned
104	17:00:56	DESC order_items	6 row(s) returned
105	17:03:26	DESC customers	8 row(s) returned
106	17:04:59	INSERT INTO products VALUES (209, 'Test Product', 'Electronics', 'TestBrand', -50, 100)	Error Code: 3819. Check constraint 'products_chk_1' is violated.
107	17:05:02	INSERT INTO products VALUES (209, 'Test Product', 'Electronics', 'TestBrand', -50, 100)	Error Code: 3819. Check constraint 'products_chk_1' is violated.

Explanation :

The unit_price column has the constraint:

CHECK (unit_price > 0)

Negative prices are not allowed.

Section B – Filtering & Optimization

Q7. Retrieve all orders with status = 'Delivered'.

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 (e_c... x

File Edit View Query Database Server Tools Scripting Help

Navigator: sql_queries* SQL File 1* celebal_week2.sample - superst...

SCHEMAS

Filter objects

- celebal_week2
 - Tables
 - Views
 - Stored Procedures
 - Functions
- e_commerce_sales_database
 - Tables
 - Views
 - Stored Procedures
 - Functions
- my_first_db
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sys
 - Tables
 - Views
 - Stored Procedures
 - Functions

143
144
145 • INSERT INTO products
146 VALUES (
147 209,
148 'Test Product',
149 'Electronics',
150 'TestBrand',
151 -50,
152 100
153);
154
155 • SELECT *
156 FROM orders
157 WHERE status = 'Delivered';

Result Grid

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1008	103	2024-08-20	Delivered	899.00
1010	108	2024-08-28	Delivered	1598.00

Schema: celebal_week2

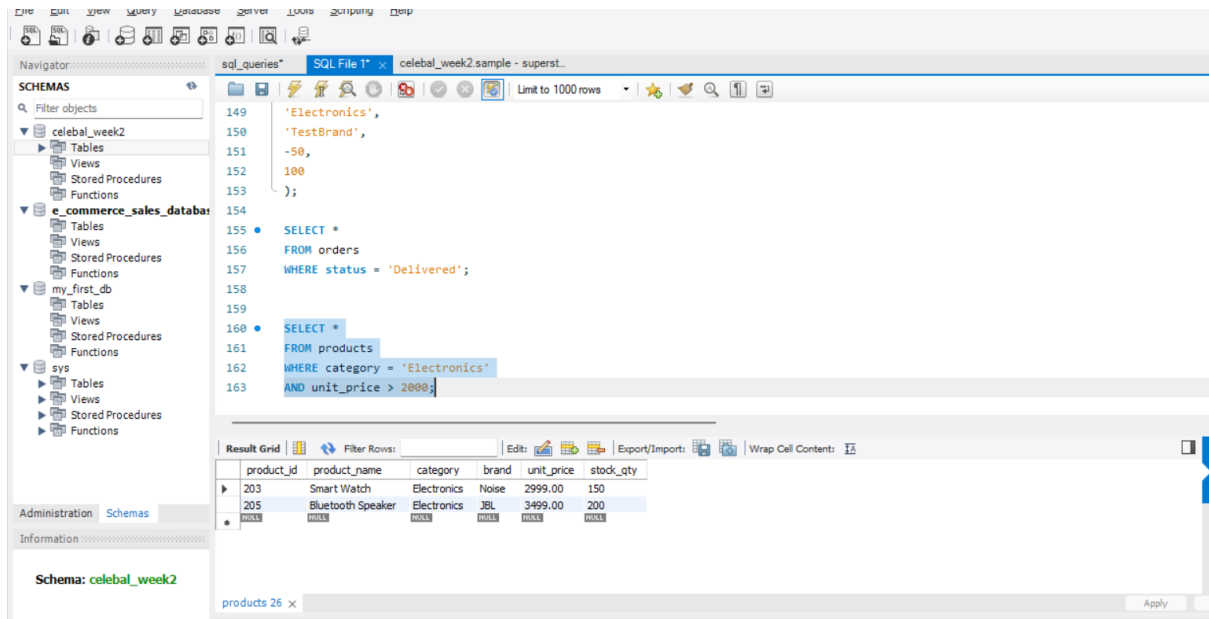
Output

Action Output

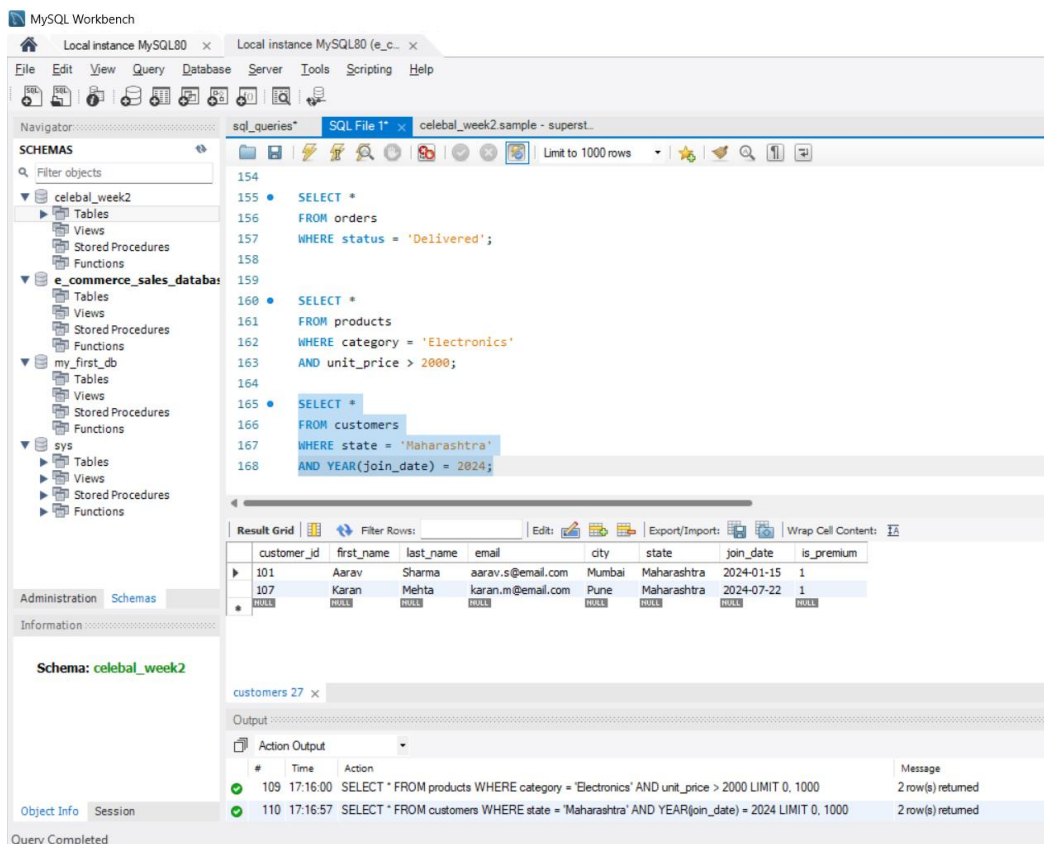
#	Time	Action	Message
103	17:00:56	DESC orders	5 row(s) returned
104	17:00:56	DESC order_items	6 row(s) returned

Query Completed

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.



Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.



Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the 'celebal_week2' database and the 'e_commerce_sales_database'. The main editor window contains the following SQL query:

```

159
160 SELECT *
161 FROM products
162 WHERE category = 'Electronics'
163 AND unit_price > 2000;
164
165 SELECT *
166 FROM customers
167 WHERE state = 'Maharashtra'
168 AND YEAR(join_date) = 2024;
169
170 SELECT *
171 FROM orders
172 WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
173 AND status <> 'Cancelled';

```

The 'Result Grid' shows the following data:

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00

The 'Output' pane shows the execution of the query, indicating that 2 rows were returned for the first query and 5 rows for the second query.

Q11. Explain what the index idx_orders_date does.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the 'celebal_week2' database and the 'e_commerce_sales_database'. The main editor window contains the following SQL query:

```

163 AND unit_price > 2000;
164
165 SELECT *
166 FROM customers
167 WHERE state = 'Maharashtra'
168 AND YEAR(join_date) = 2024;
169
170 SELECT *
171 FROM orders
172 WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
173 AND status <> 'Cancelled';
174
175 SELECT *
176 FROM orders
177 WHERE order_date BETWEEN '2024-08-01' AND '2024-08-31';

```

The 'Result Grid' shows the following data:

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498.00
1002	102	2024-08-03	Delivered	799.00
1003	103	2024-08-05	Shipped	7498.00
1004	101	2024-08-10	Delivered	3499.00
1005	104	2024-08-12	Cancelled	2999.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00

The 'Output' pane shows the execution of the query, indicating that 2 rows were returned for the first query and 5 rows for the second query.

Q12. Would the index be used in the query below?

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 (e.c. x

File Edit View Query Database Server Tools Scripting Help

Navigator: Filter objects

SCHEMAS

- celebal_week2
 - Tables
 - Views
 - Stored Procedures
 - Functions
- e_commerce_sales_database
 - Tables
 - Views
 - Stored Procedures
 - Functions
- my_first_db
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sys
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

Schema: celebal_week2

sql_queries* SQL File 1* x celebal_week2 sample - superst...

Limit to 1000 rows

```

167 WHERE state = 'Maharashtra'
168 AND YEAR(join_date) = 2024;
169
170 • SELECT *
171 FROM orders
172 WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
173 AND status <> 'Cancelled';
174
175 • SELECT *
176 FROM orders
177 WHERE order_date BETWEEN '2024-08-01' AND '2024-08-31';
178
179 • SELECT *
180 FROM customers
181 WHERE YEAR(join_date) = 2024;

```

Result Grid

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

customers 30 x

Output

Action Output

#	Time	Action	Message
112	17:18:49	SELECT * FROM orders WHERE order_date BETWEEN '2024-08-01' AND '2024-08-31' LIMIT 0, 1000	10 row(s) returned
113	17:19:33	SELECT * FROM customers WHERE YEAR(join_date) = 2024 LIMIT 0, 1000	8 row(s) returned

Object Info Session

Query Completed

Section C – Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 (e.c. x

File Edit View Query Database Server Tools Scripting Help

Navigator: Filter objects

SCHEMAS

- celebal_week2
 - Tables
 - Views
 - Stored Procedures
 - Functions
- e_commerce_sales_database
 - Tables
 - Views
 - Stored Procedures
 - Functions
- my_first_db
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sys
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

Schema: celebal_week2

sql_queries* SQL File 1* x celebal_week2 sample - superst...

Limit to 1000 rows

```

170 • SELECT *
171 FROM orders
172 WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
173 AND status <> 'Cancelled';
174
175 • SELECT *
176 FROM orders
177 WHERE order_date BETWEEN '2024-08-01' AND '2024-08-31';
178
179 • SELECT *
180 FROM customers
181 WHERE YEAR(join_date) = 2024;
182
183 • SELECT COUNT(*) AS total_orders
184 FROM orders;

```

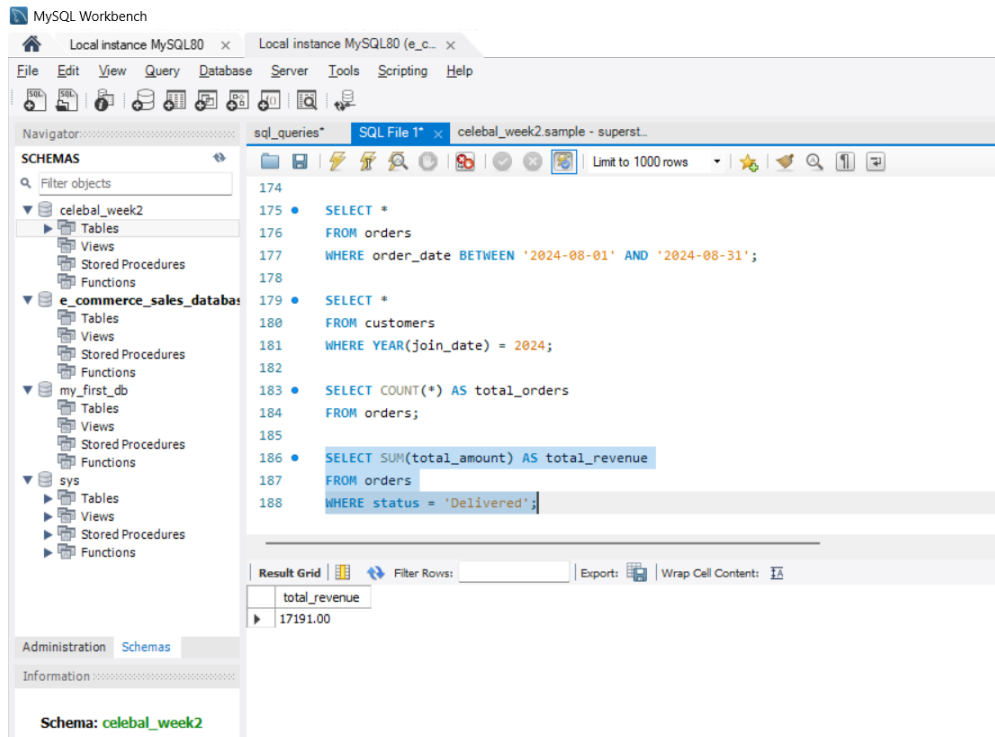
Result Grid

total_orders
10

Result 31 x

Output

Q14. Find the total revenue (SUM of total_amount) from all Delivered orders.



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' panel with a tree view of databases: 'celebal_week2', 'e_commerce_sales_databases', 'my_first_db', and 'sys'. The 'celebal_week2' database is selected. The main editor window shows a SQL query file named 'celebal_week2 sample - superst...'. The query is as follows:

```
174
175 • SELECT *
176 FROM orders
177 WHERE order_date BETWEEN '2024-08-01' AND '2024-08-31';
178
179 • SELECT *
180 FROM customers
181 WHERE YEAR(join_date) = 2024;
182
183 • SELECT COUNT(*) AS total_orders
184 FROM orders;
185
186 • SELECT SUM(total_amount) AS total_revenue
187 FROM orders
188 WHERE status = 'Delivered';
```

The query results are displayed in the 'Result Grid' at the bottom. The grid shows a single row with the column 'total_revenue' and the value '17191.00'.

total_revenue
17191.00

The bottom status bar indicates the current schema is 'celebal_week2'.

Q15. Calculate the average unit_price of products in each category.

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 (e_c... x

File Edit View Query Database Server Tools Scripting Help

Navigator: sql_queries* SQL File 1* x celebal_week2.sample - superst...

Limit to 1000 rows

```

179 • SELECT *
180 FROM customers
181 WHERE YEAR(join_date) = 2024;
182
183 • SELECT COUNT(*) AS total_orders
184 FROM orders;
185
186 • SELECT SUM(total_amount) AS total_revenue
187 FROM orders
188 WHERE status = 'Delivered';
189
190 • SELECT category,
191       AVG(unit_price) AS average_price
192 FROM products
193 GROUP BY category;

```

Result Grid

category	average_price
Electronics	2224.000000
Clothing	2699.000000
Home	949.000000

Administration Schemas

Information

Schema: celebal_week2

Object Info Session

Result 33 x

Output

Action Output

#	Time	Action	Message
115	17:23:31	SELECT SUM(total_amount) AS total_revenue FROM orders WHERE status = 'Delivered' LIMIT 0, 1000	1 row(s) returned
116	17:24:18	SELECT category, AVG(unit_price) AS average_price FROM products GROUP BY category LIMIT 0, 1000	3 row(s) returned

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 (e_c... x

File Edit View Query Database Server Tools Scripting Help

Navigator: sql_queries* SQL File 1* x celebal_week2.sample - superst...

Limit to 1000 rows

```

187 FROM orders
188 WHERE status = 'Delivered';
189
190 • SELECT category,
191       AVG(unit_price) AS average_price
192 FROM products
193 GROUP BY category;
194
195
196 • SELECT status,
197       COUNT(*) AS total_orders,
198       SUM(total_amount) AS total_revenue
199 FROM orders
200 GROUP BY status
201 ORDER BY total_revenue DESC;

```

Result Grid

status	total_orders	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	1	1299.00

Administration Schemas

Information

Schema: celebal_week2

Result 34 x

Output

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 (e_c... x

File Edit View Query Database Server Tools Scripting Help

Navigator: Filter objects

SCHEMAS

- celebal_week2
 - Tables
 - Views
 - Stored Procedures
 - Functions
- e_commerce_sales_databases
 - Tables
 - Views
 - Stored Procedures
 - Functions
- my_first_db
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sys
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

Schema: celebal_week2

sql_queries* SQL File 1* x celebal_week2.sample - superst...

Limit to 1000 rows

```

189
190 • SELECT category,
191       AVG(unit_price) AS average_price
192 FROM products
193 GROUP BY category;
194
195
196 • SELECT status,
197       COUNT(*) AS total_orders,
198       SUM(total_amount) AS total_revenue
199 FROM orders
200 GROUP BY status
201 ORDER BY total_revenue DESC;
202
203 • SELECT category,
204       MAX(unit_price) AS highest_price,

```

Result Grid

category	highest_price	lowest_price
Electronics	3499.00	899.00
Clothing	4599.00	799.00
Home	1299.00	599.00

Export: Wrap Cell Content: I

Result 35 x

Output

Q18. List all product categories where the average unit_price is greater than ₹2000.

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 (e_c... x

File Edit View Query Database Server Tools Scripting Help

Navigator: Filter objects

SCHEMAS

- celebal_week2
 - Tables
 - Views
 - Stored Procedures
 - Functions
- e_commerce_sales_databases
 - Tables
 - Views
 - Stored Procedures
 - Functions
- my_first_db
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sys
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

Schema: celebal_week2

sql_queries* SQL File 1* x celebal_week2.sample - superst...

Limit to 1000 rows

```

199 FROM orders
200 GROUP BY status
201 ORDER BY total_revenue DESC;
202
203 • SELECT category,
204       MAX(unit_price) AS highest_price,
205       MIN(unit_price) AS lowest_price
206 FROM products
207 GROUP BY category;
208
209 • SELECT category,
210       AVG(unit_price) AS average_price
211 FROM products
212 GROUP BY category
213 HAVING AVG(unit_price) > 2000;

```

Result Grid

category	average_price
Electronics	2224.000000
Clothing	2699.000000

Export: Wrap Cell Content: I

Result 36 x

Output

Section D – Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 (e_c_ x

File Edit View Query Database Server Tools Scripting Help

Navigator: sql_queries* SQL File 1* celebal_week2.sample - superst_

SCHEMAS

Filter objects

- celebal_week2
 - Tables
 - Views
 - Stored Procedures
 - Functions
- e_commerce_sales_databas
 - Tables
 - Views
 - Stored Procedures
 - Functions
- my_first_db
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sys
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

Schema: celebal_week2

```

209 • SELECT category,
210         AVG(unit_price) AS average_price
211 FROM products
212 GROUP BY category
213 HAVING AVG(unit_price) > 2000;
214
215
216 • SELECT o.order_id,
217         o.order_date,
218         c.first_name,
219         c.last_name,
220         o.total_amount
221 FROM orders o
222 INNER JOIN customers c
223 ON o.customer_id = c.customer_id;
  
```

Result Grid

	order_id	order_date	first_name	last_name	total_amount
▶	1001	2024-08-01	Aarav	Sharma	4498.00
	1004	2024-08-10	Aarav	Sharma	3499.00
	1002	2024-08-03	Priya	Patel	799.00
	1003	2024-08-05	Rohan	Gupta	7498.00
	1008	2024-08-20	Rohan	Gupta	899.00
	1005	2024-08-12	Sneha	Reddy	2999.00
	1006	2024-08-15	Vikram	Singh	5898.00

Result 37 x

Output

Action Output

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any).

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 (e_c_ x

File Edit View Query Database Server Tools Scripting Help

Navigator: sql_queries* SQL File 1* celebal_week2.sample - superst_

SCHEMAS

Filter objects

- celebal_week2
 - Tables
 - Views
 - Stored Procedures
 - Functions
- e_commerce_sales_databas
 - Tables
 - Views
 - Stored Procedures
 - Functions
- my_first_db
 - Tables
 - Views
 - Stored Procedures
 - Functions
- sys
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

Schema: celebal_week2

```

219         c.last_name,
220         o.total_amount
221 FROM orders o
222 INNER JOIN customers c
223 ON o.customer_id = c.customer_id;
224
225
226 • SELECT c.customer_id,
227         c.first_name,
228         c.last_name,
229         o.order_id,
230         o.order_date
231 FROM customers c
232 LEFT JOIN orders o
233 ON c.customer_id = o.customer_id;
  
```

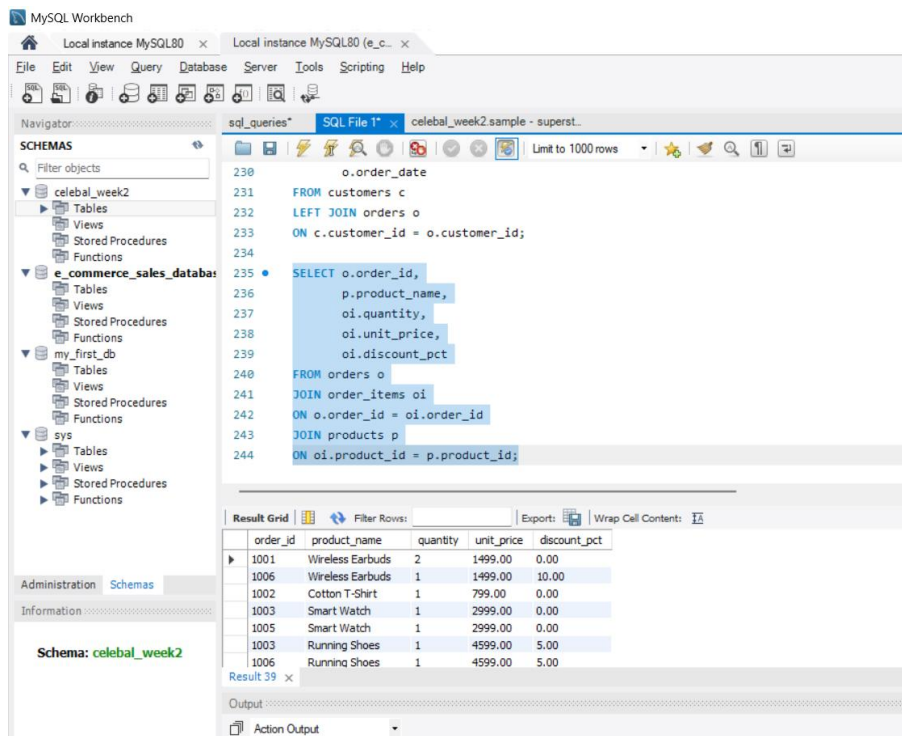
Result Grid

	customer_id	first_name	last_name	order_id	order_date
▶	101	Aarav	Sharma	1001	2024-08-01
	101	Aarav	Sharma	1004	2024-08-10
	102	Priya	Patel	1002	2024-08-03
	103	Rohan	Gupta	1003	2024-08-05
	103	Rohan	Gupta	1008	2024-08-20
	104	Sneha	Reddy	1005	2024-08-12
	105	Vikram	Singh	1006	2024-08-15

Result 38 x

Output

Q21. Write a query using JOINS across three tables (orders → order_items → products).



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

230 o.order_date
231 FROM customers c
232 LEFT JOIN orders o
233 ON c.customer_id = o.customer_id;
234
235 • SELECT o.order_id,
236        p.product_name,
237        oi.quantity,
238        oi.unit_price,
239        oi.discount_pct
240 FROM orders o
241 JOIN order_items oi
242 ON o.order_id = oi.order_id
243 JOIN products p
244 ON oi.product_id = p.product_id;

```

The Results tab shows the following data:

order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499.00	0.00
1006	Wireless Earbuds	1	1499.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1005	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1006	Running Shoes	1	4599.00	5.00

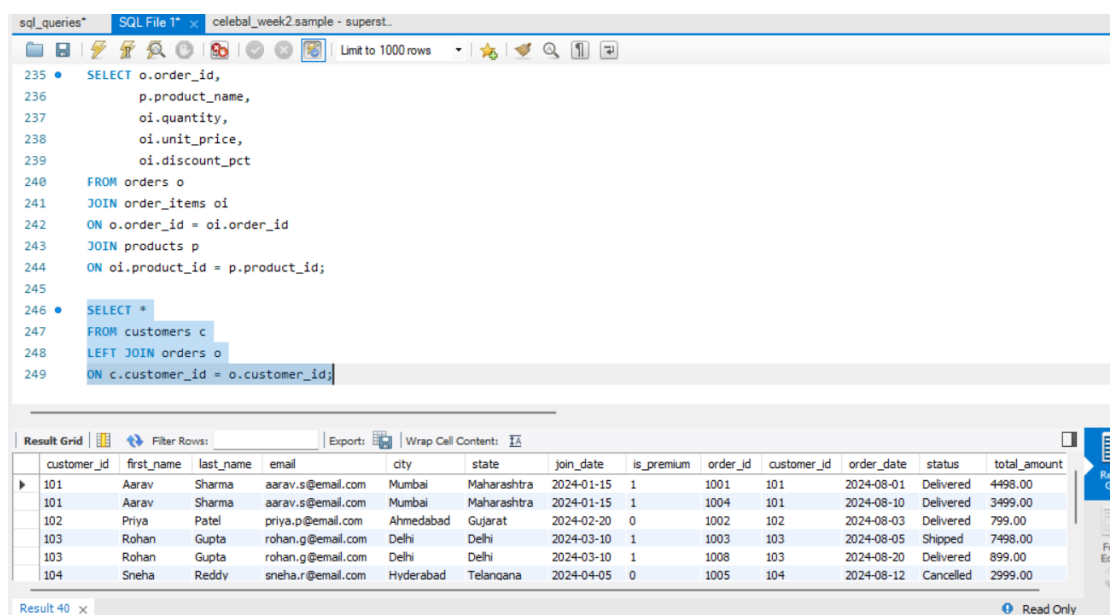
Q22. Explain the difference between LEFT JOIN and RIGHT JOIN.

LEFT JOIN

```

SELECT *
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;

```



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

235 • SELECT o.order_id,
236        p.product_name,
237        oi.quantity,
238        oi.unit_price,
239        oi.discount_pct
240 FROM orders o
241 JOIN order_items oi
242 ON o.order_id = oi.order_id
243 JOIN products p
244 ON oi.product_id = p.product_id;
245
246 • SELECT *
247 FROM customers c
248 LEFT JOIN orders o
249 ON c.customer_id = o.customer_id;

```

The Results tab shows the following data:

customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	order_date	status	total_amount
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024-08-01	Delivered	4498.00
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	2024-08-10	Delivered	3499.00
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	2024-08-03	Delivered	799.00
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	2024-08-05	Shipped	7498.00
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	2024-08-20	Delivered	899.00
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024-08-12	Cancelled	2999.00

Right Join :

The screenshot shows a SQL IDE with a query window and a result grid. The query is as follows:

```
240 FROM orders o
241 JOIN order_items oi
242 ON o.order_id = oi.order_id
243 JOIN products p
244 ON oi.product_id = p.product_id;
245
246 • SELECT *
247 FROM customers c
248 LEFT JOIN orders o
249 ON c.customer_id = o.customer_id;
250
251 • SELECT *
252 FROM customers c
253 RIGHT JOIN orders o
254 ON c.customer_id = o.customer_id;
```

The result grid displays the following data:

	customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	order_date	status	total_amount
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024-08-01	Delivered	4498.00
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-01-15	0	1002	102	2024-08-03	Delivered	799.00
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-02-10	0	1003	103	2024-08-05	Shipped	7498.00
	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	2024-08-10	Delivered	3499.00
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024-08-12	Cancelled	2999.00
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	2024-08-15	Delivered	5898.00

Q23. Identify all Foreign Key relationships in the schema.

The screenshot shows a SQL IDE with a query window and an output window. The query is as follows:

```
241 JOIN order_items oi
242 ON o.order_id = oi.order_id
243 JOIN products p
244 ON oi.product_id = p.product_id;
245
246 • SELECT *
247 FROM customers c
248 LEFT JOIN orders o
249 ON c.customer_id = o.customer_id;
250
251 • SELECT *
252 FROM customers c
253 RIGHT JOIN orders o
254 ON c.customer_id = o.customer_id;
255
256
257 • INSERT INTO orders
258 VALUES (
259 1011,
260 999,
261 '2024-09-01',
262 'Pending',
263 1000
264 );
265
```

The output window shows the following error message:

```
127 18:24:55 INSERT INTO orders VALUES ( 1011, 999, '2024-09-01', 'Pending', 1000 )
Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('e_commerce_sales_databa...

128 18:27:00 INSERT INTO orders VALUES ( 1011, 999, '2024-09-01', 'Pending', 1000 )
Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('e_commerce_sales_databa...
```

Explanation

Customer ID 999 does not exist in the customers table. Therefore, the Foreign Key constraint prevents the insertion.

Section E – Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers.

The screenshot shows a SQL IDE with a query editor and a result grid. The query in the editor is as follows:

```
260 999,  
261 '2024-09-01',  
262 'Pending',  
263 1000  
264 );  
265  
266 • SELECT product_name,  
267         unit_price,  
268         CASE  
269             WHEN unit_price < 1000 THEN 'Budget'  
270             WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'  
271             ELSE 'Premium'  
272         END AS price_tier  
273 FROM products;  
274
```

The result grid below the query shows the following data:

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-Range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-Range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-Range
Laptop Stand	899.00	Budget

Result 42 x

Output :

Q25. Using a CASE statement inside an aggregate function, count how many orders are Delivered vs Not Delivered.

The screenshot shows a SQL IDE with a query editor and a result grid. The query in the editor is as follows:

```
265  
266 • SELECT product_name,  
267         unit_price,  
268         CASE  
269             WHEN unit_price < 1000 THEN 'Budget'  
270             WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'  
271             ELSE 'Premium'  
272         END AS price_tier  
273 FROM products;  
274  
275  
276 • SELECT  
277     SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS Delivered_Orders,  
278     SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS Not_Delivered_Orders  
279 FROM orders;
```

The result grid below the query shows the following data:

Delivered_Orders	Not_Delivered_Orders
6	4

Q26. Explain ACID Properties.

A – Atomicity

Atomicity ensures that all operations in a transaction are completed successfully. If any operation fails, the entire transaction is rolled back.

Example: During a bank transfer, if money is deducted from one account but cannot be added to another account, the transaction is cancelled completely.

C – Consistency

Consistency ensures that the database remains in a valid state before and after a transaction.

Example: After a bank transfer, the total balance across accounts remains accurate and follows all database rules.

I – Isolation

Isolation ensures that multiple transactions do not interfere with each other while executing simultaneously.

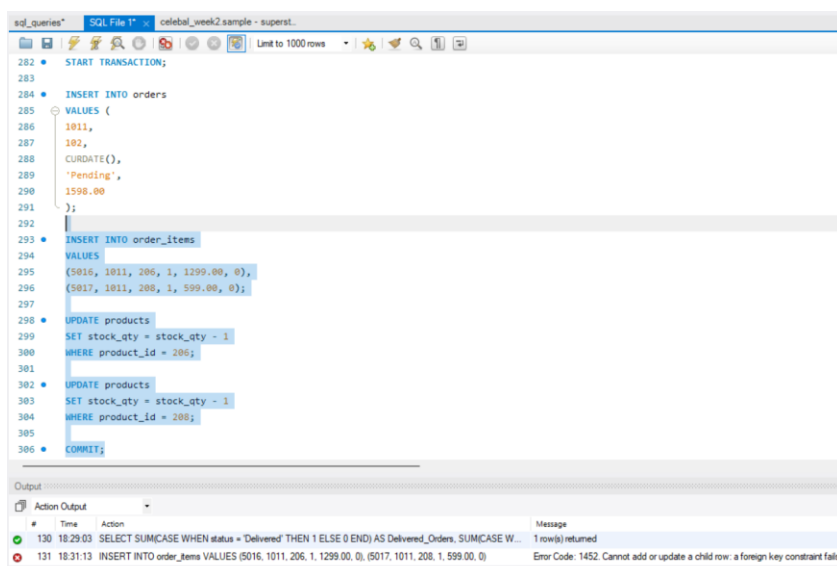
Example: Two users transferring money at the same time should not affect each other's transaction results.

D – Durability

Durability ensures that once a transaction is committed, the data remains permanently stored even if the system crashes.

Example: After a successful money transfer, the updated account balances remain saved even after a power failure.

Q27. Write a SQL transaction that performs the following atomically.



```
sql_queries*  SQL File 1*  celebal_week2.sample - superal.
282 • START TRANSACTION;
283
284 • INSERT INTO orders
285   VALUES (
286     1011,
287     102,
288     CURDATE(),
289     'Pending',
290     1598.00
291   );
292
293 • INSERT INTO order_items
294   VALUES
295   (5016, 1011, 206, 1, 1299.00, 0),
296   (5017, 1011, 208, 1, 599.00, 0);
297
298 • UPDATE products
299   SET stock_qty = stock_qty - 1
300   WHERE product_id = 206;
301
302 • UPDATE products
303   SET stock_qty = stock_qty - 1
304   WHERE product_id = 208;
305
306 • COMMIT;
```

Output

#	Time	Action	Message
130	18:29:03	SELECT SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS Delivered_Orders, SUM(CASE W...	1 row(s) returned
131	18:31:13	INSERT INTO order_items VALUES (5016, 1011, 206, 1, 1299.00, 0), (5017, 1011, 208, 1, 599.00, 0)	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (

Explanation

The transaction failed because one of the SQL statements generated an error. Since transactions follow the Atomicity rule, all operations must be completed successfully. If any step fails, the database automatically rolls back the entire transaction to prevent incomplete or incorrect data from being stored.

Key Learnings and Insights

During this activity, I learned how to design and manage a relational database using SQL. I created multiple tables, established relationships using Primary Keys and Foreign Keys, and inserted data while maintaining data integrity.

I also gained practical experience with SQL concepts such as filtering, aggregation, joins, indexes, CASE statements, and transactions. These queries helped in analyzing customer orders, product performance, and sales information.

Challenges Faced and Solutions

- While creating tables, I had to ensure that parent tables were created before child tables to avoid Foreign Key errors.
- During data insertion, I carefully followed the table relationships to prevent constraint violations.
- Some queries generated errors due to constraints, which helped me understand the importance of Primary Keys, Foreign Keys, and CHECK constraints.
- While working with transactions, I learned how ROLLBACK protects the database from incomplete or failed operations.